# FedAvg vs FedProx 비교 실험 보고서

Non-IID (Dirichlet α=0.3) CIFAR-10 · Flower · SuperLink(AWS EC2 / Oracle Cloud A1) + Mac / Windows Desktop SuperNode · 2026-11-09~10 실행

> **요약.** 클라이언트 2개, 10라운드 조건에서 μ를 0.01, 0.1, 1.0으로 바꿔 FedProx를 FedAvg와 비교했다. **FedProx(μ=0.01)가 가장 좋았다.** 최종 정확도는 μ=0.01 54.97%, FedAvg 52.38%, μ=0.1 41.72%, μ=1.0 27.20%였다. μ=0.01은 FedAvg보다 2.59%p 높았고, 최저점 대비 손실 상승도 8.4%로 FedAvg(25.3%)보다 작았다. 반대로 μ가 커질수록 학습이 느려져 정확도가 크게 떨어졌다. 즉 이 설정에서 근접항은 **아주 약하게 줄 때만** 도움이 됐다. 단, 각 조건을 시드 1회씩만 실행했기 때문에 2-3%p 차이는 우연일 수 있다.

## 1. 실험 목적

클라이언트마다 라벨 분포가 크게 다른 Non-IID 환경에서, 근접항(proximal term)을 추가한 FedProx가 FedAvg보다 수렴과 안정성이 나은지 확인한다. 또 근접항의 세기 μ에 따라 결과가 어떻게 달라지는지 본다.

## 2. 실험 환경 및 설정

| 항목 | 값 |
|---|---|
| 프레임워크 | Flower Deployment Engine ( `flwr run` ), 한 프로젝트( `fedcompare` )에서 run-config로 전략과 μ만 바꿈 |
| 구성 | SuperNode 2개: Windows Desktop(partition 0), Mac(partition 1) · Tailscale 연결<br>SuperLink: FedAvg, μ=0.1은 AWS EC2 · μ=0.01, μ=1.0은 Oracle Cloud A1(2 OCPU, ARM) |
| 데이터 | CIFAR-10 학습셋 50,000장을 Dirichlet(α=0.3, seed=42)로 2분할하고, 노드마다 80% 학습 / 20% 로컬 평가로 나눔 |
| 모델 | 간단한 CNN (Conv 2층 + FC 3층, PyTorch 60 Minute Blitz 기반), 초기 가중치 seed=42 고정 |
| 로컬 학습 | SGD (lr=0.01, momentum=0.9), batch 32, local epoch 3 |
| 연합 학습 | 10라운드, 매 라운드 2개 노드 전원 참여 (fraction-evaluate=1.0) |
| 비교 대상 | FedAvg (μ=0) vs FedProx (μ=0.01, 0.1, 1.0). 손실에 $(\mu/2)\cdot \| w - w_{global} \|^2$를 더함 |
| 평가 | 서버: CIFAR-10 test 10,000장 전역 평가 · 클라이언트: 로컬 20% 평가셋의 가중 평균 |
| 소요 시간 | FedAvg 7분 03초 · μ=0.1 6분 57초 (AWS) · μ=0.01 10분 18초 · μ=1.0 10분 16초 (Oracle) |

SuperLink 서버가 실험 중간에 바뀌었지만 코드, 시드, 설정은 같다. 네 실험 모두 0라운드(초기 모델) 정확도가 10.2%로 같아서 같은 초기 가중치에서 출발했음을 확인했다. Oracle 실행이 약 3분 더 걸린 것은 서버 사양 차이로, 학습 결과에는 영향이 없다.

### 데이터 분포 (Non-IID 정도)

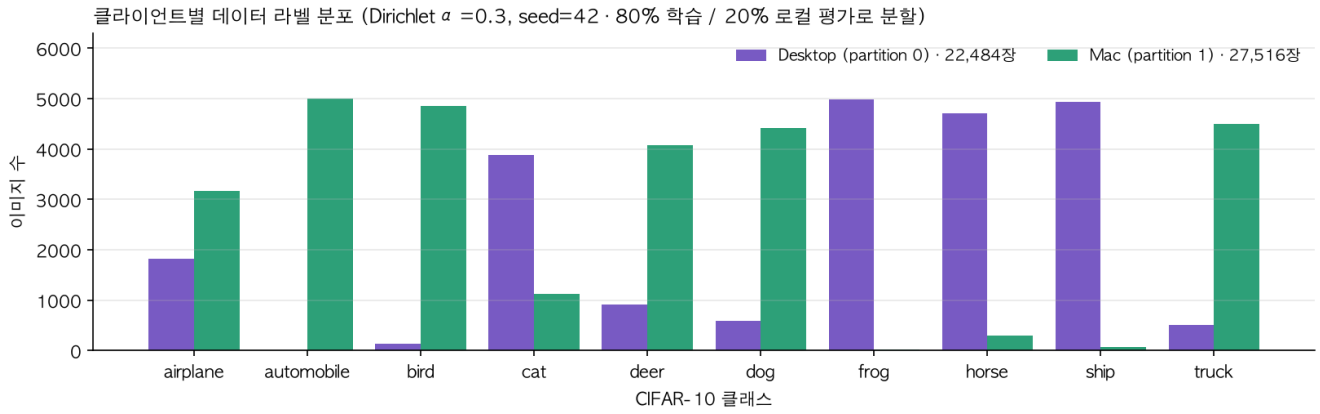



그림 1. 두 노드의 라벨 분포가 거의 겹치지 않는다. Desktop은 frog·horse·ship·cat 위주이고 automobile은 4장뿐이다. Mac은 automobile·bird·truck·dog 위주이고 frog 22장, ship 69장, horse 295장으로 이 세 클래스가 거의 없다. 노드 하나만으로는 10개 클래스를 고르게 학습할 수 없는 강한 Non-IID 조건이다.

## 3. 결과

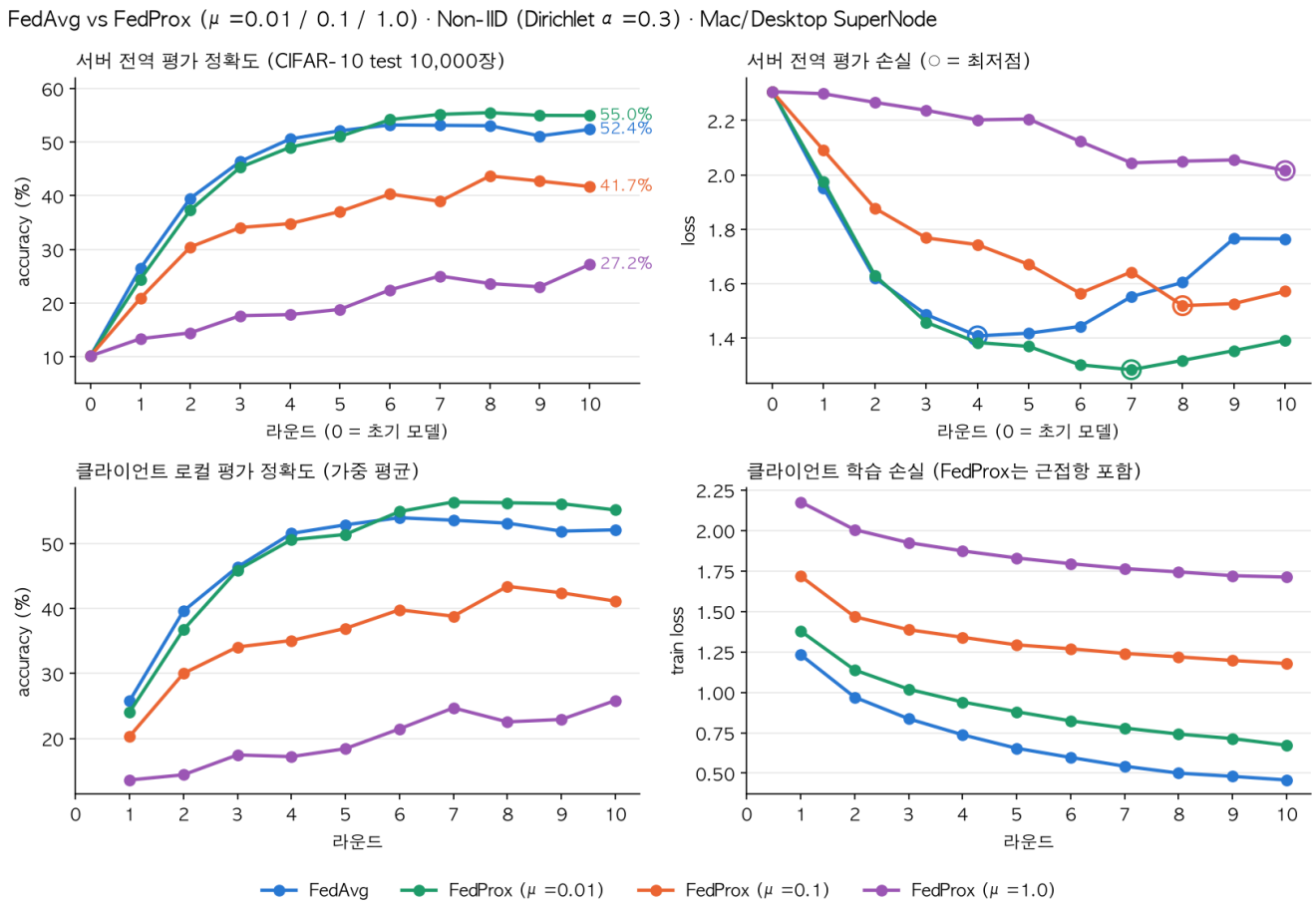



그림 2. 라운드별 서버 전역 정확도와 손실, 클라이언트 로컬 평가 정확도, 클라이언트 학습 손실

| 지표 (서버 전역 평가) | FedAvg | FedProx μ=0.01 | FedProx μ=0.1 | FedProx μ=1.0 |
|---|---|---|---|---|
| 최종 정확도 (R10) | 52.38% | **54.97%** | 41.72% | 27.20% |
| 최고 정확도 | 53.21% (R6) | **55.50% (R8)** | 43.70% (R8) | 27.20% (R10) |
| R6-R10 평균 정확도 | 52.59% | **54.97%** | 41.49% | 24.28% |
| R6-R10 표준편차 (흔들림) | 0.78%p | **0.43%p** | 1.69%p | 1.69%p |
| 40% / 50% 도달 라운드 | **R3 / R4** | R3 / R5 | R6 / 못함 | 못함 / 못함 |
| 최저 손실 | 1.408 (R4) | **1.284 (R7)** | 1.519 (R8) | 2.016 (R10) |
| 최종 손실 (R10) | 1.765 | **1.392** | 1.572 | 2.016 |
| 최저점 대비 최종 손실 상승률 | +25.3% | +8.4% | +3.5% | 0% (아직 감소 중) |
| 클라이언트 로컬 평가 최종 정확도 | 52.10% | **55.16%** | 41.14% | 25.85% |

## 4. 분석

- **μ=0.01이 정확도와 안정성을 모두 잡았다.** 초반 속도는 FedAvg와 거의 같았다 (40% 도달 R3, 50% 도달은 FedAvg보다 한 라운드 늦은 R5). 6라운드부터 FedAvg를 앞섰고, 마지막 5라운드 평균은 54.97%로 FedAvg보다 2.38%p 높았다. 흔들림(표준편차)도 0.43%p로 가장 작았다.
- **FedAvg는 4라운드 이후 클라이언트 드리프트로 보이는 신호를 보였다.** 정확도는 52-53%에서 정체됐는데 서버 손실은 1.408(R4)에서 1.765(R10)로 25% 올랐다. 틀린 예측에 대한 확신이 커졌다는 뜻이다. μ=0.01은 같은 지표가 8.4%에 그쳐, 약한 근접항만으로도 로컬 모델이 자기 데이터 쪽으로 치우치는 것을 상당 부분 막았다.
- **μ가 커질수록 학습이 느려졌다.** μ=0.1은 10라운드 안에 50%에 못 갔고, μ=1.0은 27%에 머물렀다. 근접항이 너무 세면 각 노드가 전역 모델에서 거의 움직이지 못해서 라운드당 학습량이 크게 줄어든다. μ=1.0은 손실이 끝까지 내려가는 중이라, 라운드를 늘리면 정확도는 더 오르겠지만 수렴이 매우 느리다.
- **손실 안정성과 정확도 사이에 트레이드오프가 있다.** 손실 상승률만 보면 μ가 클수록 작다 (25.3% → 8.4% → 3.5% → 0%). 하지만 μ=0.1부터는 안정성을 얻는 대가로 정확도를 크게 잃는다. 이 설정에서는 μ=0.01이 그 균형점에 가장 가까웠다.
- **학습 손실 그래프는 직접 비교할 수 없다.** FedProx의 train loss에는 근접항이 더해져 있어서 μ가 클수록 원래 높게 나온다. 그림 2 오른쪽 아래 그래프의 차이를 덜 학습됐다는 근거로 쓰면 안 된다.

## 5. 한계

- 조건마다 시드 1회, 실행 1회 결과라서 분산을 알 수 없다. μ=0.01과 FedAvg의 2-3%p 차이는 우연일 수 있다. 반면 μ=0.1, 1.0과의 10%p 이상 차이는 우연으로 보기 어렵다.
- SuperLink 서버가 실험 도중 AWS에서 Oracle Cloud로 바뀌었다. 코드, 시드, 초기 정확도는 같지만 완전히 같은 환경은 아니다.
- 클라이언트가 2개뿐이고 매 라운드 전원 참여했다. FedProx가 원래 노리는 시스템 이질성(노드마다 다른 학습량, 부분 참여, 지연 노드)은 다루지 않았다.
- 모델이 작고 라운드 수가 적어서 정확도 절대값이 낮다 (최고 55.5%).

## 6. 결론 및 다음 실험

이번 설정(노드 2개, 전원 참여, 10라운드)에서는 **FedProx(μ=0.01)가 FedAvg보다 최종 정확도가 2.59%p 높고 손실도 더 안정적이었다.** μ=0.1 이상은 근접항이 너무 세서 정확도가 크게 떨어졌다. 따라서 이 모델에서 FedProx를 쓴다면 μ를 0.01 근처의 작은 값으로 잡아야 한다.

- 시드 3개 이상으로 FedAvg와 μ=0.01을 반복해서, 2-3%p 차이가 실제 차이인지 평균과 표준편차로 확인한다.
- μ=0.001, 0.005, 0.02, 0.05를 추가해서 최적 μ 구간을 더 좁힌다.
- 라운드를 30회 이상으로 늘려서 FedAvg의 드리프트가 정확도 하락으로 이어지는지 본다.
- 노드별 local epoch를 다르게 주는(예: 1 vs 5) 시스템 이질성 조건을 추가한다.

원자료: `~/FL/NonIID/results/noniid_fedavg.json`, `noniid_fedprox_mu0.01.json`, `noniid_fedprox_mu0.1.json`, `noniid_fedprox_mu1.json` · 그래프 코드: `~/FL/NonIID/plot_compare.py` · 라벨 분포: `fedcompare.task.label_hist`로 재계산